1) Discuss the synchronization issues that may arise and justify the mechanisms you selected to address them.

The main problem in our program was that in our implementation of the producer and consumer model, all threads had to access the same buffer, which with improper implementation would cause problems since no thread knows what the other threads do.

Our solution was based on the implementation of conditions and locks to synchronize buffer access. For instance, no two threads can read or write at the same time in the buffer due to a lock. We would also want producers to wait until there is an empty slot in the buffer to add a new element and for consumers to wait until there is an element in the buffer to process it. Both of these tasks are achieved by using conditions that threads use to wake up other threads when either an empty slot is present or an element has been added to the buffer.

2) Explain why your solution prevents race conditions, starvation, and busy waiting.

- Prevent race conditions: As mentioned previously, each thread has access to a lock, which prevents multiple threads from accessing the same resources

- Prevent starvation: In order to avoid an unfair distribution of resources, tasks that take more time but do not require accessing critical resources are done outside the locked portion of the code. In our case the processing of the data is done separately from writing and accessing the buffer. This way we make sure that each thread does not spend more time with the lock than it needs to.

- Prevent busy waiting: Using conditions, we can make threads wait while they cannot do anything. This way we prevent them from using additional computational resources when they are not doing nothing.

3) Describe how the program behaves since the first producer reaches EOF until the process ends, paying special attention to consumers that may still be blocked waiting for data. Note that when a producer reaches the EOF, there might be other producers which are still doing them.

When a producer reaches EOF it means that the next time any producer attempts to read that file, it will read 0 bytes, which, when it does, will wake up any waiting consumer, it will change the state of the variable producersFinished from 0 to 1 and it will exit.

Note: there is a record of how many bytes each element in the buffer contains, so no consumer will read more bytes than the element has.

After consumers are woken up, they are going to retrieve and process the remaining elements in the buffer. Every time a consumer checks that the number of elements in the buffer is 0 it will check for the variable producersFinished. If it's 1, it exits; if not, it waits until it receives the signal from the producers. When a consumer exits, it also returns its own histogram, which the main thread is going to use to compute the total histogram.